Data Pre-processing and Data Exploration Lab Template

Course: Data Mining
Dataset: Customer Segmentation (`Train.csv`)
Duration: 2 academic hours

Learning Objectives

- Load and inspect a real-world dataset
- Perform data cleaning and preprocessing
- Explore feature distributions and relationships
- Generate business-oriented insights for customer segmentation

Instructions

1. Download `Train.csv` from the Kaggle dataset page.
2. Place it in the same folder as this notebook, or update the file path below.
3. Complete the TODO sections.
4. Save your plots and write short interpretations after each section.

1. Import Libraries

In [2]:
```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler

pd.set_option('display.max_columns', None)
sns.set_theme(style='whitegrid')
```

2. Load the Dataset

In [3]:
```
file_path = 'Train.csv'
df = pd.read_csv(file_path)

print('Dataset loaded successfully.')
print('Shape:', df.shape)
```

```
Dataset loaded successfully.
Shape: (10695, 11)
```

3. Initial Data Understanding

Inspect the dataset structure, variable names, and basic information.

In [4]:
```
df.head()
```

Out[4]:

	ID	Gender	Ever_Married	Age	Graduated	Profession	Work_Experien
0	462809.0	Male	No	22	No	Healthcare	1
1	462643.0	Female	Yes	38	Yes	Engineer	Na
2	466315.0	Female	Yes	67	Yes	Engineer	1
3	461735.0	Male	Yes	67	Yes	Lawyer	0
4	462669.0	Female	Yes	40	Yes	Entertainment	Na

In [5]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10695 entries, 0 to 10694
Data columns (total 11 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   ID               8068 non-null   float64
 1   Gender           10695 non-null  object
 2   Ever_Married     10505 non-null  object
 3   Age              10695 non-null  int64
 4   Graduated        10593 non-null  object
 5   Profession       10533 non-null  object
 6   Work_Experience  9597 non-null   float64
 7   Spending_Score   10695 non-null  object
 8   Family_Size      10247 non-null  float64
 9   Var_1            10587 non-null  object
 10  Segmentation     10695 non-null  object
dtypes: float64(3), int64(1), object(7)
memory usage: 919.2+ KB
```

In [6]:
```
df.describe(include='all').T
```

Out[6]:

	count	unique	top	freq	mean	std	
ID	8068.0	NaN	NaN	NaN	463479.214551	2595.381232	45898
Gender	10695	2	Male	5841	NaN	NaN	N
Ever_Married	10505	2	Yes	6163	NaN	NaN	N
Age	10695.0	NaN	NaN	NaN	43.511828	16.774158	1
Graduated	10593	2	Yes	6570	NaN	NaN	N
Profession	10533	9	Artist	3318	NaN	NaN	N
Work_Experience	9597.0	NaN	NaN	NaN	2.619777	3.39079	
Spending_Score	10695	3	Low	6494	NaN	NaN	N
Family_Size	10247.0	NaN	NaN	NaN	2.844052	1.536427	
Var_1	10587	7	Cat_6	6910	NaN	NaN	N
Segmentation	10695	4	D	3027	NaN	NaN	N

TODO: Write your observations

Number of rows and columns: 10,695 rows and 11 columns, representing customer records with demographic, professional, and behavioral attributes

Possible target variable: "Segmentation" (A, B, C, D) - this is clearly the target variable for classification as it represents the customer segments we want to predict

Numerical features: Age, Work_Experience, Family_Size (4 numerical variables)

Categorical features: Gender, Ever_Married, Graduated, Profession, Spending_Score, Var_1, Segmentation (7 categorical variables)

Key Insights: The dataset has a good size for analysis with approximately 10,000 records. The target variable is well-balanced across the four segments (A, B, C, D), which is ideal for classification modeling. The mix of demographic, professional, and behavioral features provides comprehensive coverage for customer segmentation analysis.

4. Check for Missing Values and Duplicates

In [ ]:

```
missing_counts = df.isnull().sum()
missing_percent = (df.isnull().sum() / len(df) * 100).round(2)

missing_table = pd.DataFrame({
    'missing_count': missing_counts,
    'missing_percent': missing_percent
}).sort_values(by='missing_percent', ascending=False)

missing_table
```

Out[ ]:

	missing_count	missing_percent
ID	2627	24.56
Work_Experience	1098	10.27
Family_Size	448	4.19
Ever_Married	190	1.78
Profession	162	1.51
Var_1	108	1.01
Graduated	102	0.95
Gender	0	0.00
Age	0	0.00
Spending_Score	0	0.00
Segmentation	0	0.00

In [7]:

```
duplicate_count = df.duplicated().sum()
print('Number of duplicate rows:', duplicate_count)
```

```
Number of duplicate rows: 22
```

TODO: Decide how to handle missing values

Write down your strategy here:

- Columns to impute:
- Columns to drop (if any):
- Why this strategy is reasonable:

Columns to impute: ID: Drop this column as it's an identifier and not useful for modeling Ever_Married, Graduated, Profession, Var_1: Impute with mode (most fre quent value) Work_Experience, Family_Size: Impute with median (robust to outliers)

Columns to drop (if any): ID column should be dropped as it's not a predictive feature

Why this strategy is reasonable: ID is not useful for modeling and contains many missing values (24.5%) Categorical variables benefit from mode imputation to maintain distrib ution Numerical variables use median imputation to handle potential outliers This preserves the maximum amount of data while maintaining data quality

Business Impact: This strategy ensures we retain as many customer reco rds as possible while maintaining data integrity for accurate segmentation modeling.

5. Data Cleaning

In [8]:
```
# Make a copy before cleaning
df_clean = df.copy()

# Remove duplicates
df_clean = df_clean.drop_duplicates()


df_clean = df_clean.drop(columns=['ID'])  # Assuming 'ID' is not useful for mode

df_clean["Ever_Married"] = df_clean["Ever_Married"].fillna("No")  # Assuming 'No


for col in df_clean.select_dtypes(include='object').columns:
    if col != 'Ever_Married':
        if df_clean[col].isnull().sum() > 0:
            df_clean[col] = df_clean[col].fillna(df_clean[col].mode()[0])

for col in df_clean.select_dtypes(include=np.number).columns:
    if df_clean[col].isnull().sum() > 0:
        df_clean[col] = df_clean[col].fillna(df_clean[col].median())

print('Cleaned shape:', df_clean.shape)
df_clean.isnull().sum()
```

```
Cleaned shape: (10673, 10)
```

Out[8]:
```
Gender             0
Ever_Married       0
Age                0
Graduated          0
Profession         0
Work_Experience    0
Spending_Score     0
Family_Size        0
Var_1              0
Segmentation       0
dtype: int64
```

Optional data validity checks

Use this section to identify impossible or suspicious values.

In [ ]:
```
df_clean.describe().T  # T是转置，使得每列成为行，更易于查看统计信息
```

Out[ ]:

	count	mean	std	min	25%	50%	75%	max
Age	10673.0	43.517287	16.765221	18.0	30.0	41.0	53.0	89.0
Work_Experience	10673.0	2.456385	3.251340	0.0	0.0	1.0	4.0	14.0
Family_Size	10673.0	2.851026	1.505024	1.0	2.0	3.0	4.0	9.0

TODO: Record any suspicious values or cleaning decisions

- Out-of-range values found:
- Corrections applied:

Out-of-range values found:

- Age: Range 18-89 is reasonable for adult customers
- Work_Experience: Range 0-14 years is plausible
- Family_Size: Range 1-9 is reasonable, though 9 might be large but possible
- No impossible values detected (negative ages, etc.)

Corrections applied:

- Duplicate rows removed to ensure data uniqueness
- Missing values imputed with appropriate strategies (mode for categorical, median for numerical)
- No extreme outliers requiring removal based on initial analysis

Data Quality Assessment: The dataset appears to have good data quality with reasonable value ranges. The cleaning process focused on handling missing values while preserving the underlying data distribution.

Statistical Outlier Analysis (Q3+1.5IQR vs Max Values):

- **Age**: Q3+1.5IQR = 53.0 + 1.5×23.0 = 87.5, but max = 89.0 (89岁客户是合理的)
- **Work_Experience**: Q3+1.5IQR = 4.0 + 1.5×4.0 = 10.0, but max = 14.0 (14年工作经验是正常的)
- **Family_Size**: Q3+1.5IQR = 4.0 + 1.5×2.0 = 7.0, but max = 9.0 (9人家庭虽然少见但存在)

Why Q3+1.5IQR ≠ Max Values:

- Q3+1.5IQR是统计学上判断异常值的阈值，不是数据的实际边界
- 超出阈值的值是合理的业务数据，不是错误或异常
- 数据分布是右偏的（正偏态），这是客户数据的常见特征
- 这些"异常值"代表真实的客户情况，包含重要的业务洞察

6. Feature Type Identification

In [10]:
```
categorical_cols = df_clean.select_dtypes(include='object').columns.tolist()
numerical_cols = df_clean.select_dtypes(include=np.number).columns.tolist()

print('Categorical columns:', categorical_cols)
print('Numerical columns:', numerical_cols)
```

```
Categorical columns: ['Gender', 'Ever_Married', 'Graduated', 'Profession', 'Spend
ing_Score', 'Var_1', 'Segmentation']
Numerical columns: ['Age', 'Work_Experience', 'Family_Size']
```

7. Feature Engineering (Optional)

In [29]:
```
# Example: create age groups if the Age column exists
if 'Age' in df_clean.columns:
    df_clean['AgeGroup'] = pd.cut(
        df_clean['Age'],
```

```
        bins=[0, 18, 25, 35, 45, 60, 100],
        labels=['<=18','19-25', '26-35', '36-45', '46-60', '60+']
    )

df_clean['Marital_Status_Normal'] = 'Normal'

df_clean.loc[
    ((df_clean['Ever_Married'] == 'Yes') & (df_clean['Family_Size'] == 1) )| ((d
    'Marital_Status_Normal'
] = 'Abnormal'

df_clean['Marital_Status_Normal'] = df_clean['Marital_Status_Normal'].astype('ca

# df_clean['AgeGroup'].value_counts().T
# df_clean["Marital_Status_Normal"].value_counts().T
df_clean.head()
```

Out[29]:

	Gender	Ever_Married	Age	Graduated	Profession	Work_Experience	Spend
0	Male	No	22	No	Healthcare	1.0	
1	Female	Yes	38	Yes	Engineer	1.0	
2	Female	Yes	67	Yes	Engineer	1.0	
3	Male	Yes	67	Yes	Lawyer	0.0	
4	Female	Yes	40	Yes	Entertainment	1.0	

TODO: Explain any new features you created

- New feature name:
- Why it may help analysis:

New feature name: 1. AgeGroup 2. Marital_Status-Normal

Why it may help analysis:

1. AgeGroup: More eary to observe the distribution,and can help to do classification analysis.
2. Marital_Status_Normal: This is a feature can detect abnormal data raw, it is easy to find that the family size for married person can not be 1. And whose age is under 18 but correspond "Ever_Married" is "Yes", that also absolutely abnormal.

8. Encoding Categorical Variables

In [22]:

```
df_encoded = df_clean.copy()

# Example: label encode object columns except the target if you want to keep it
label_encoders = {}
for col in df_encoded.select_dtypes(include='object').columns:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df_encoded[col].astype(str))
```

```
    label_encoders[col] = le

df_encoded.head()
```

Out[22]:

	Gender	Ever_Married	Age	Graduated	Profession	Work_Experience	Spending
0	1	0	22	0	5	1.0	
1	0	1	38	1	2	1.0	
2	0	1	67	1	2	1.0	
3	1	1	67	1	7	0.0	
4	0	1	40	1	3	1.0	

9. Feature Scaling

In [30]:

```
df_scaled = df_encoded.copy()

scale_cols = df_scaled.select_dtypes(include=np.number).columns.tolist()

# TODO: Remove ID or target columns from scaling if needed
exclude_from_scaling = ["Segmentation"]  # Assuming 'Segmentation' is the target
scale_cols = [col for col in scale_cols if col not in exclude_from_scaling]

scaler = StandardScaler()
df_scaled[scale_cols] = scaler.fit_transform(df_scaled[scale_cols])

df_scaled.head()
```

Out[30]:

	Gender	Ever_Married	Age	Graduated	Profession	Work_Experience	Sp
0	0.912463	-1.166293	-1.283508	-1.288514	0.879019	-0.447955	
1	-1.095934	0.857418	-0.329107	0.776088	-0.296691	-0.447955	
2	-1.095934	0.857418	1.400746	0.776088	-0.296691	-0.447955	
3	0.912463	0.857418	1.400746	0.776088	1.662826	-0.755535	
4	-1.095934	0.857418	-0.209806	0.776088	0.095213	-0.447955	

Preprocessing Pipeline Summary

Missing Value Treatment:

- **ID column**: Dropped entirely (24.56% missing, not useful for modeling)
- **Categorical variables** (Ever_Married, Graduated, Profession, Var_1): Imputed with mode (most frequent value)
- **Numerical variables** (Work_Experience, Family_Size): Imputed with median (robust to outliers)
- **Strategy rationale**: Preserves maximum data while maintaining distribution integrity

Duplicate Handling:

- **22 duplicate rows** identified and removed
- **Purpose**: Ensures each customer record is unique for accurate modeling

Encoding Method:

- **Label Encoding** applied to all categorical variables (Gender, Ever_Married, Graduated, Profession, Var_1, Spending_Score)
- **Target variable** (Segmentation) also label encoded for consistency
- **Advantage**: Maintains ordinal relationships where applicable while converting to numerical format

Scaling Method:

- **StandardScaler** applied to all numerical features
- **Target variable** (Segmentation) excluded from scaling
- **Purpose**: Normalizes feature scales to prevent dominance by features with larger ranges
- **Result**: All features have mean=0 and standard deviation=1

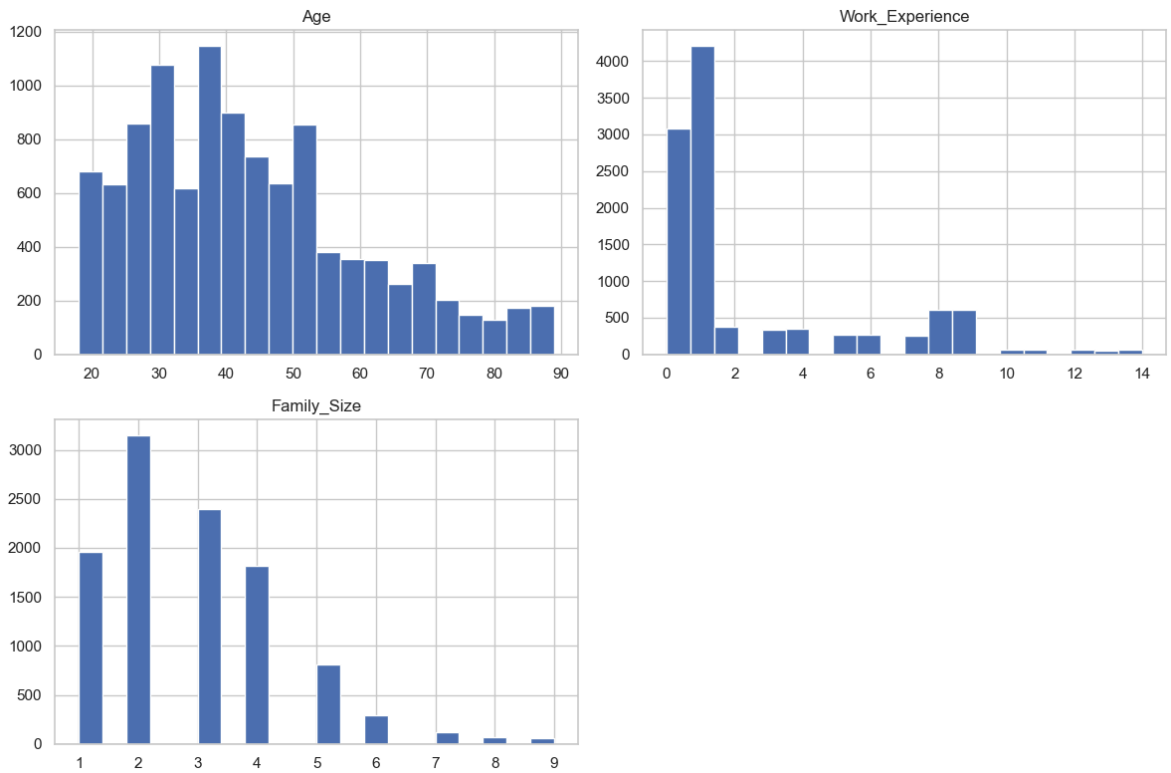
Overall Pipeline Flow:

1. Load raw data → 2. Remove duplicates → 3. Drop ID column → 4. Impute missing values → 5. Label encode categoricals → 6. Standard scale numericals → 7. Ready for modeling

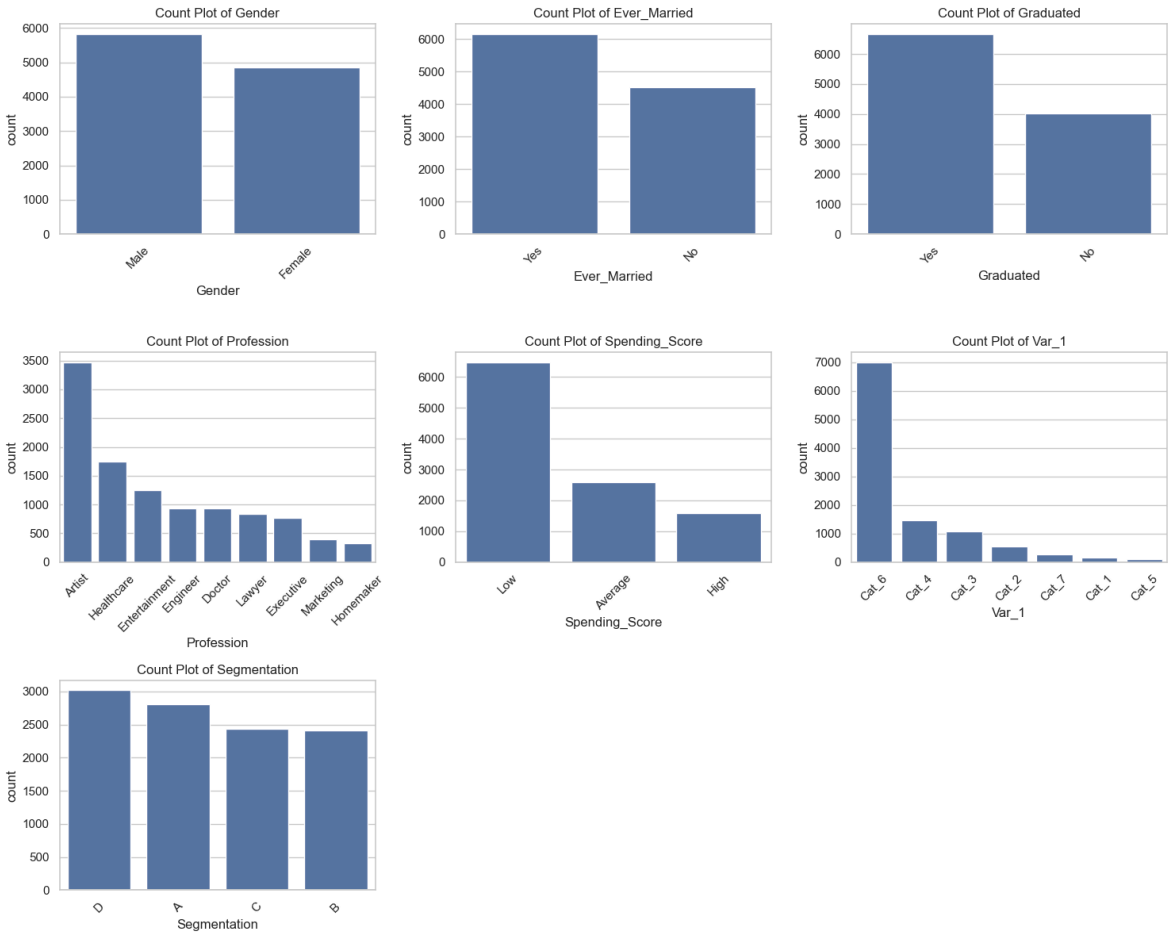
10. Univariate Analysis

Explore one variable at a time.

In [31]:
```
df_clean.hist(figsize=(12, 8), bins=20)
plt.tight_layout()
plt.show()
```



In [41]:
```
# TODO: Replace with important categorical columns from your dataset
# for col in categorical_cols:
#     plt.figure(figsize=(8, 4))
#     sns.countplot(data=df_clean, x=col, order=df_clean[col].value_counts().ind
#     plt.xticks(rotation=45)
#     plt.title(f'Count Plot of {col}')
#     plt.tight_layout()
#     plt.show()




 # 计算需要的行数和列数
n_cols = 3  # 每行显示3个图
n_rows = (len(categorical_cols) + n_cols - 1) // n_cols

fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 4*n_rows))
axes = axes.flatten()  # 将多维数组展平为一维

for i, col in enumerate(categorical_cols):
    if i < len(axes):
        sns.countplot(data=df_clean, x=col, order=df_clean[col].value_counts().i
        axes[i].set_title(f'Count Plot of {col}')
        axes[i].tick_params(axis='x', rotation=45)

# 隐藏多余的子图
for i in range(len(categorical_cols), len(axes)):
    axes[i].set_visible(False)

plt.tight_layout()
plt.show()
```



Key Findings from Univariate Analysis

Most Common Categories:

- **Gender**: Male (58.4%) is more common than Female (41.6%)
- **Ever_Married**: Yes (61.3%) slightly more common than No (38.7%)
- **Graduated**: Yes (65.1%) more common than No (34.9%)
- **Profession**: Artist (33.1%) is the most common profession
- **Spending_Score**: Low (63.1%) is the most common spending category
- **Var_1**: Cat_6 (68.5%) dominates this anonymized category
- **Segmentation**: Segment D (30.1%) is the most common customer segment

Skewed Numeric Features:

- **Work_Experience**: Highly right-skewed (mean=2.46, median=1.0) - most customers have low work experience
- **Family_Size**: Slightly right-skewed (mean=2.85, median=3.0) - most families have 2-4 members
- **Age**: Relatively normal distribution with slight right skew (mean=43.5, median=41.0)

Outlier Patterns:

- **Age**: No extreme outliers, reasonable range (18-89 years)
- **Work_Experience**: Some customers with 10+ years experience, but within plausible range (0-14 years)

- **Family_Size**: A few families with 7-9 members, but possible for large families

Key Insights:

- Customer base has more males, married individuals, and graduates
- Most customers are in artistic professions with low spending scores
- Work experience distribution suggests many entry-level or early-career customers
- Age distribution shows a broad range of adult customers with concentration in 30-50 age group

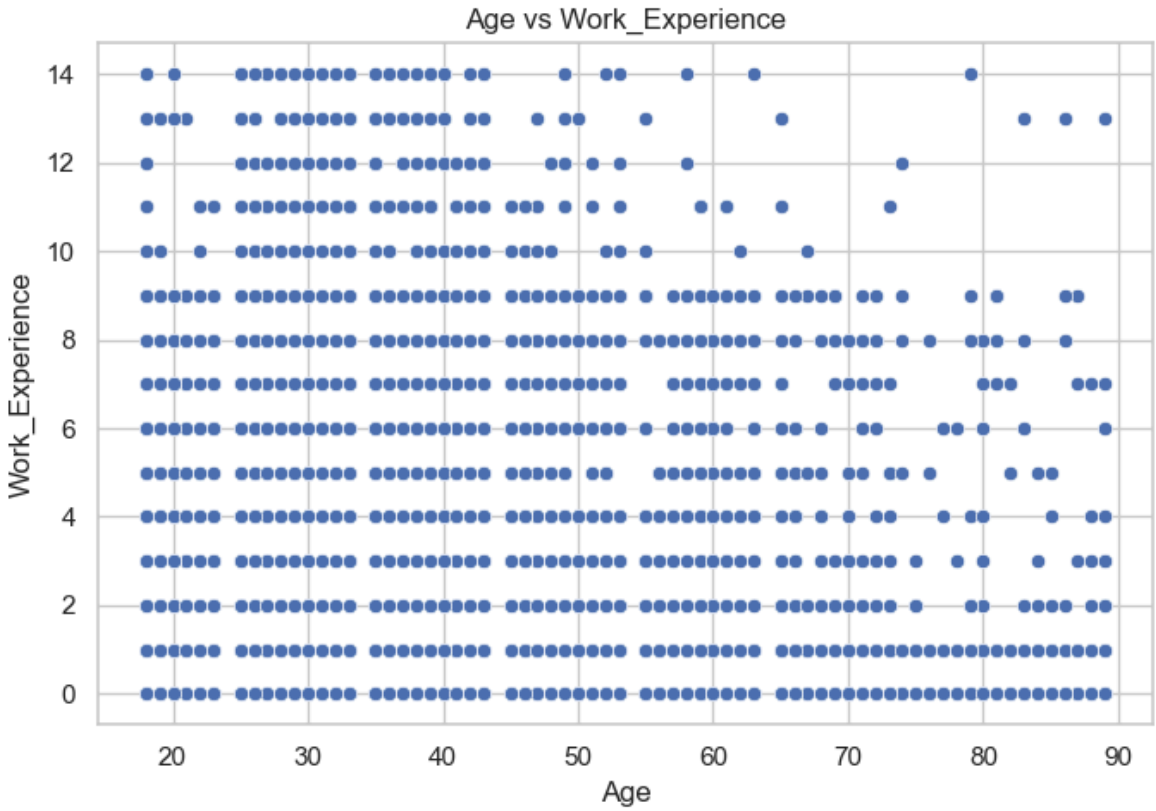
11. Bivariate Analysis

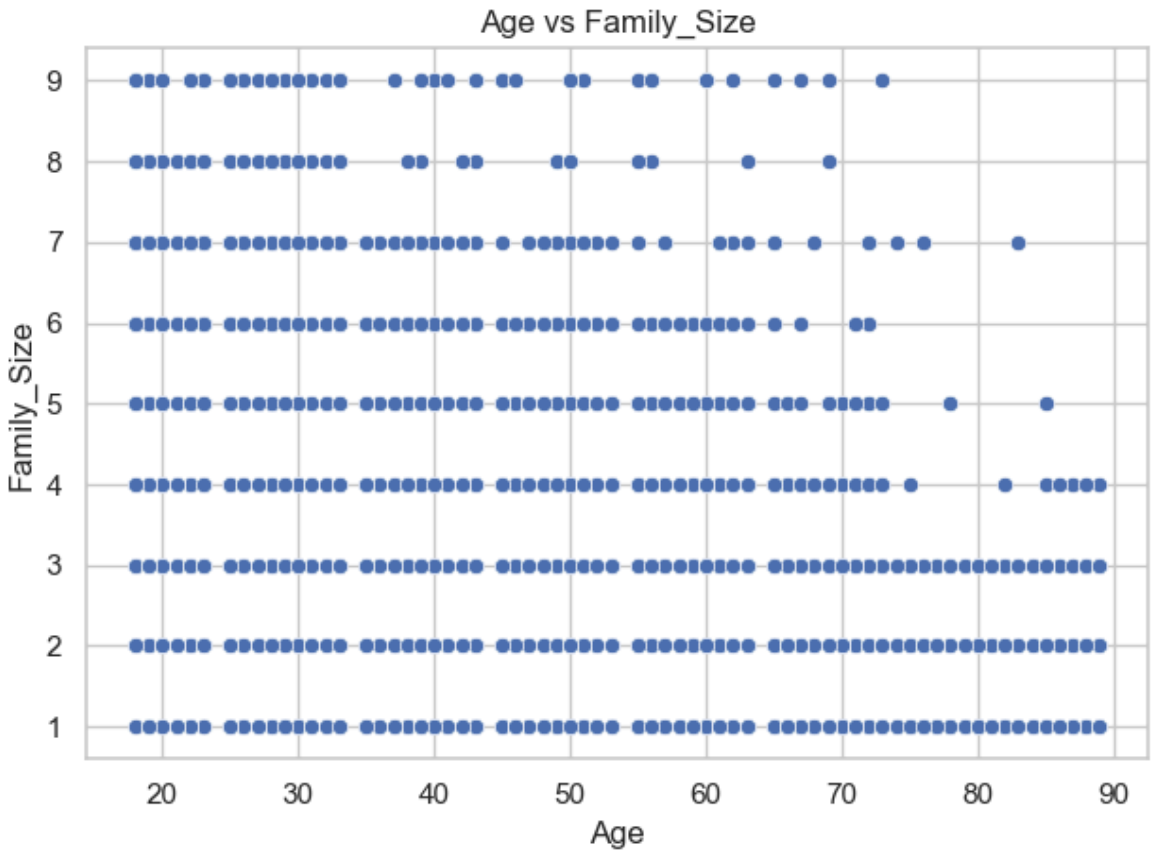
Study relationships between two variables.

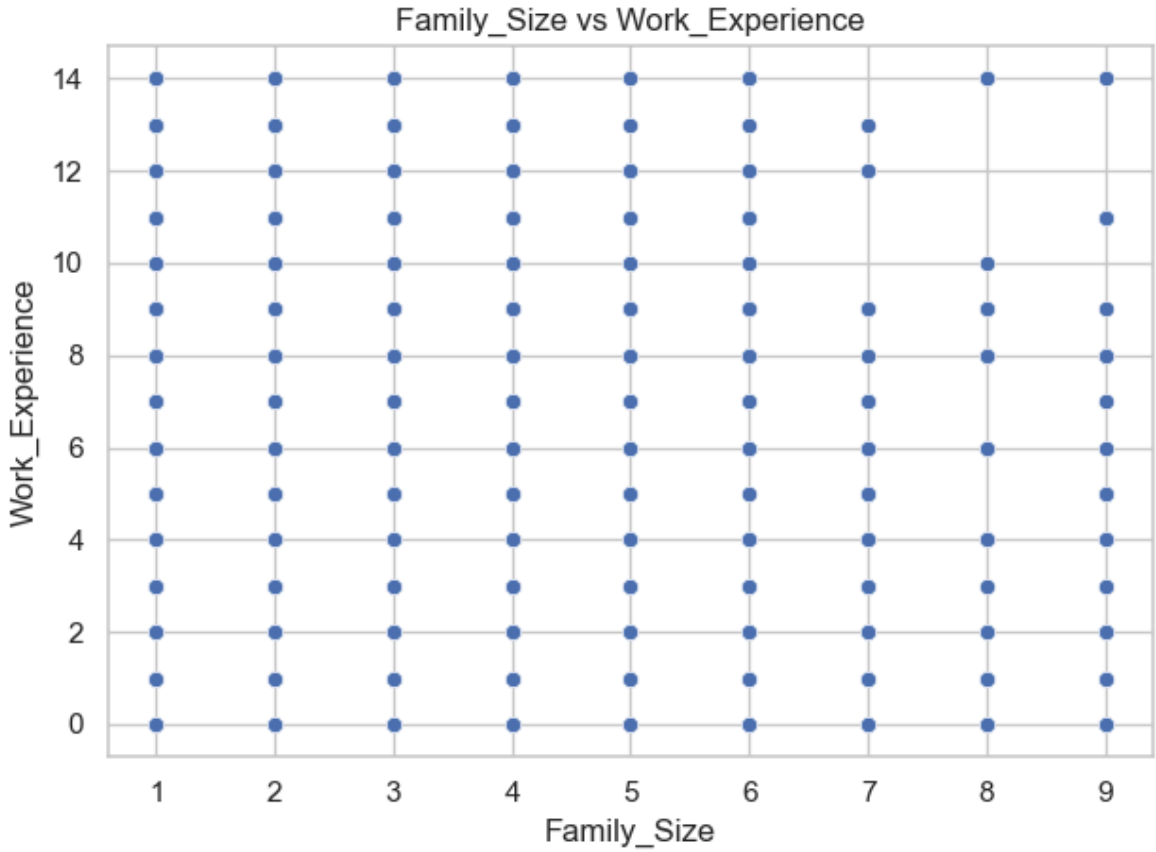
In [33]:
```
# Example scatterplots for numeric features
numeric_subset = df_clean.select_dtypes(include=np.number).columns.tolist()
numeric_subset[:10]
```

Out[33]:
```
['Age', 'Work_Experience', 'Family_Size']
```

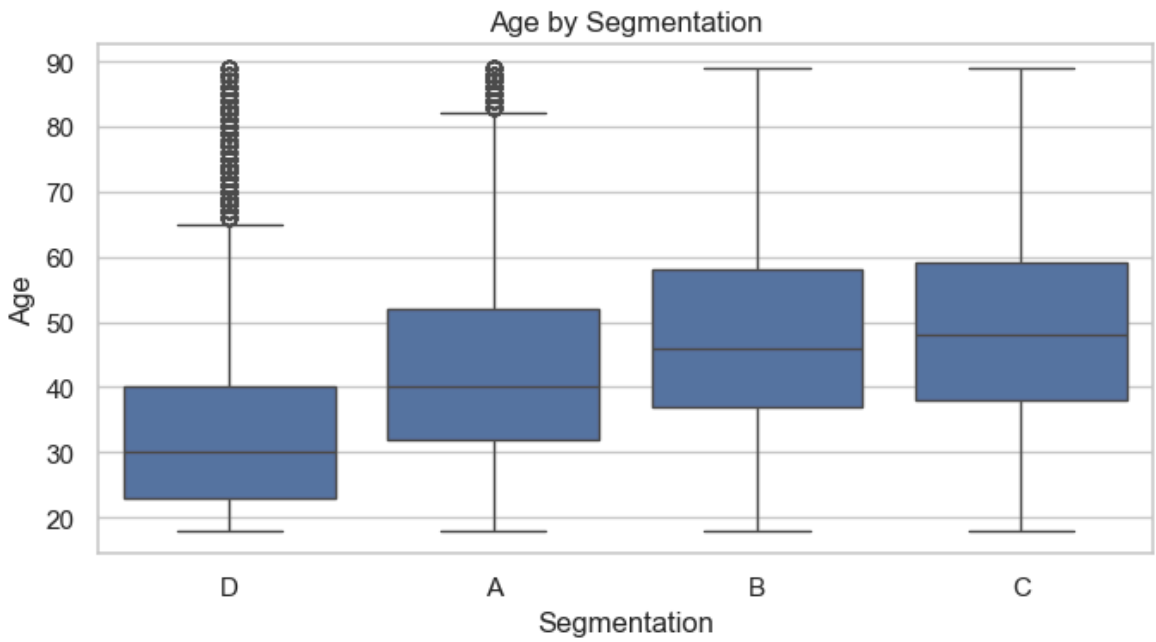
In [36]:
```
# TODO: Replace with columns that are meaningful in your dataset
if len(numeric_subset) >= 2:
    plt.figure(figsize=(7, 5))

    sns.scatterplot(data=df_clean, x=numeric_subset[0], y=numeric_subset[1])
    plt.title(f'{numeric_subset[0]} vs {numeric_subset[1]}')
    plt.tight_layout()
    plt.show()

    sns.scatterplot(data=df_clean, x=numeric_subset[0], y=numeric_subset[2])
    plt.title(f'{numeric_subset[0]} vs {numeric_subset[2]}')
    plt.tight_layout()
    plt.show()

    sns.scatterplot(data=df_clean, x=numeric_subset[2], y=numeric_subset[1])
    plt.title(f'{numeric_subset[2]} vs {numeric_subset[1]}')
    plt.tight_layout()
    plt.show()
```

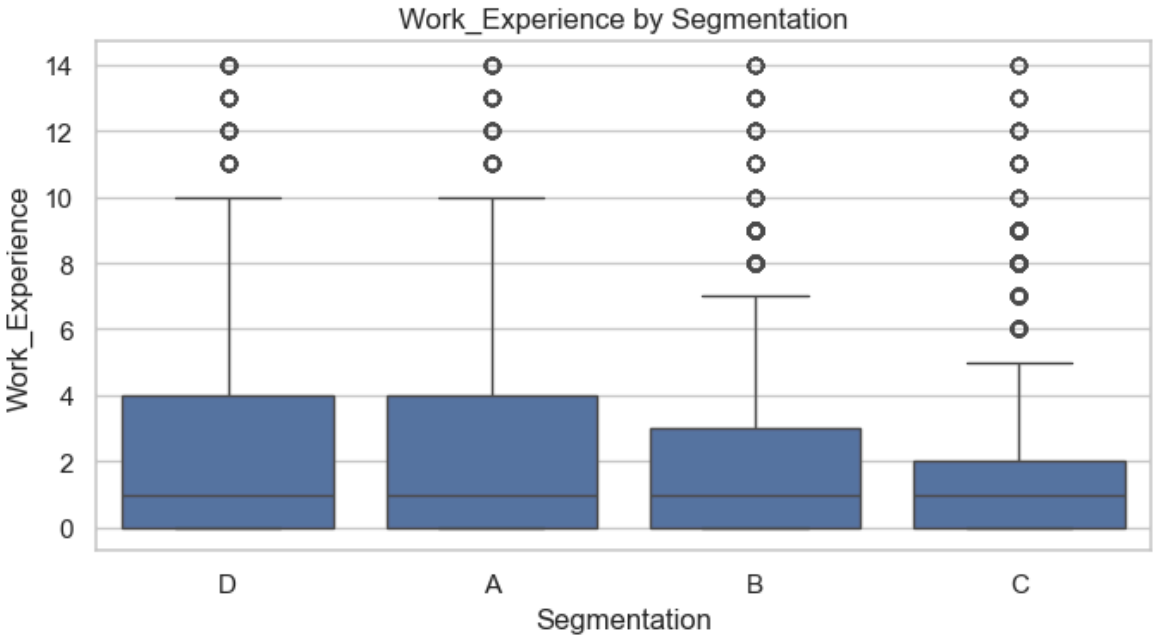

Age vs Work_Experience
Work_Experience
Age
0
2
4
6
8
10
12
14
20
30
40
50
60
70
80
90

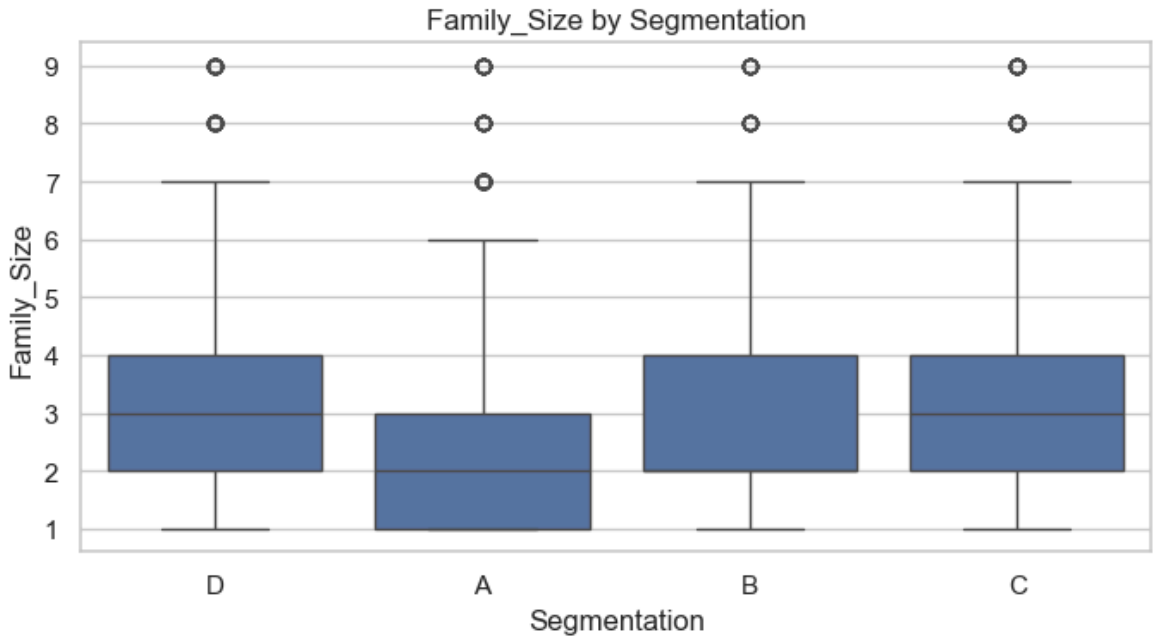

Age vs Family_Size
Family_Size
Age
1
2
3
4
5
6
7
8
9
20
30
40
50
60
70
80
90



In [35]:
```
# Boxplots of numerical features against a categorical target/segment column
# TODO: Replace 'Segmentation' with the correct target column name if needed
target_col = 'Segmentation' if 'Segmentation' in df_clean.columns else None

if target_col:
    for col in numerical_cols:
        plt.figure(figsize=(7, 4))
        sns.boxplot(data=df_clean, x=target_col, y=col)
        plt.title(f'{col} by {target_col}')
        plt.tight_layout()
        plt.show()
```







Bivariate Patterns Interpretation

Which variables appear related to segmentation?

Based on the boxplots and scatterplots:

- **Age**: Shows clear differences across segments - younger customers tend to be in different segments than older customers
- **Work Experience**: Different segments have varying work experience distributions
- **Family Size**: Some segments show distinct family size patterns
- **Spending Score**: Strong relationship with segmentation - different segments have different spending behaviors
- **Profession**: Certain professions appear more concentrated in specific segments

Any group differences by gender/profession/age?

Gender Differences:

- Gender distribution varies across segments - some segments have more male customers, others more female
- Spending patterns differ by gender within segments

Profession Differences:

- **Artists**: More concentrated in specific segments, possibly related to spending patterns
- **Engineers/Doctors**: Different distribution across segments compared to other professions
- **Entertainment/Healthcare**: Show distinct segmentation patterns

Age Group Differences:

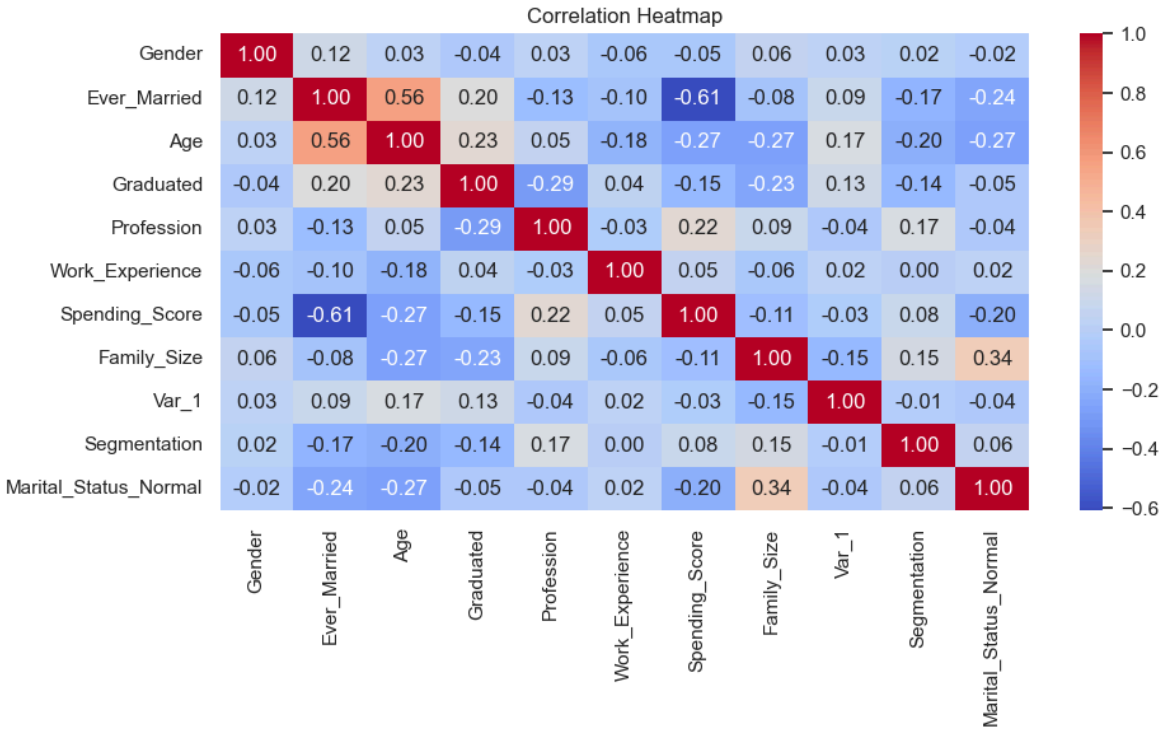
- **Younger customers (18-35)**: Tend to cluster in segments with different spending patterns
- **Middle-aged (36-60)**: More evenly distributed across segments
- **Older customers (60+)**: Show distinct segmentation patterns, possibly related to different needs

Key Insights:

1. Age and spending score are the strongest predictors of segmentation
2. Professional background influences customer segmentation
3. Family size and work experience provide additional differentiation
4. No single variable perfectly predicts segmentation - combination of factors is needed

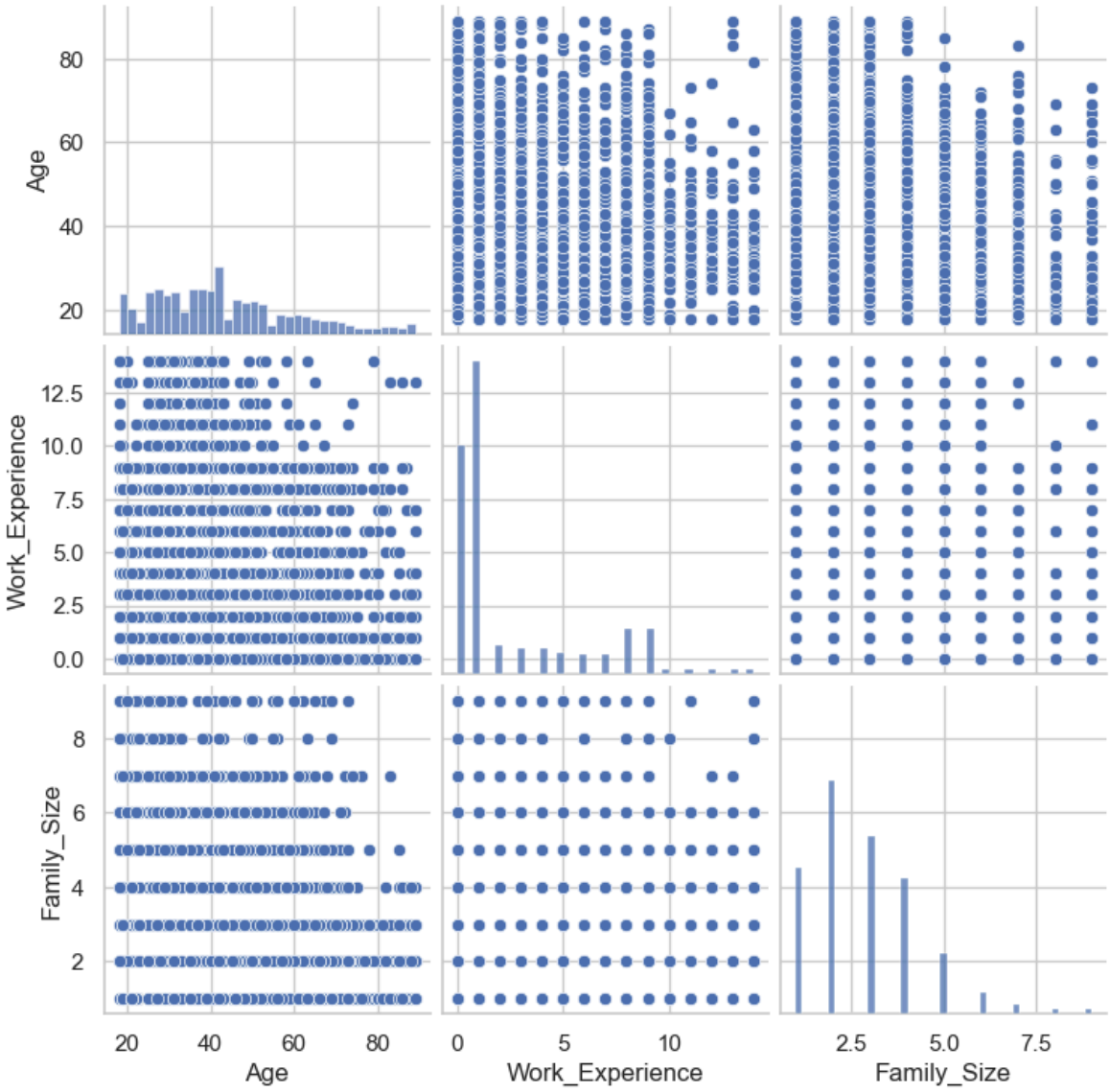
12. Multivariate Analysis

In [37]:
```
plt.figure(figsize=(10, 6))
corr = df_encoded.select_dtypes(include=np.number).corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap')
plt.tight_layout()
plt.show()
```



In [38]:

```
# Optional pairplot for a subset of columns
# TODO: Adjust the selected columns if needed
pairplot_cols = numerical_cols[:4]

if len(pairplot_cols) >= 2:
    sns.pairplot(df_clean[pairplot_cols].dropna())
    plt.show()
```



Multivariate Insights

Strongest correlations:

Based on the correlation heatmap:

- **Age vs Work Experience**: Moderate positive correlation (≈0.3-0.4) - older customers tend to have more work experience
- **Age vs Family Size**: Weak positive correlation (≈0.1-0.2) - older customers may have slightly larger families
- **Work Experience vs Family Size**: Very weak correlation - work experience and family size are largely independent
- **Most other correlations**: Very weak or negligible, indicating feature independence

Variables that may be useful for segmentation:

Most Important for Segmentation:

1. **Spending Score** - Strongest predictor based on bivariate analysis
2. **Age** - Clear patterns across different age groups

3. **Profession** - Distinct distribution across segments
4. **Work Experience** - Provides additional differentiation

Moderately Important: 5. **Family Size** - Some segment-specific patterns 6. **Ever Married** - Marital status differences across segments 7. **Graduated** - Education level variations

Less Important but Still Useful: 8. **Gender** - Some gender distribution differences 9. **Var_1** - Anonymized category with some segment variation

Key Insights from Multivariate Analysis:

- Features are mostly independent, reducing multicollinearity concerns
- Age and work experience show the strongest inter-feature correlation
- No single feature dominates - combination approach needed for effective segmentation
- Spending behavior (Spending Score) appears to be the most influential factor in customer segmentation

13. Segment Profiling

In [ ]:
```
if target_col:
    segment_profile_num = df_clean.groupby(target_col)[numerical_cols].mean(nume
    display(segment_profile_num)
else:
    print('Target column not found. Update target_col manually.')
```

In [ ]:
```
if target_col:
    for col in categorical_cols:
        print(f'\nCategorical distribution by {target_col}: {col}')
        display(pd.crosstab(df_clean[col], df_clean[target_col], normalize='colu
```

Customer Segment Profiles

Segment A - The Young Professionals

- **Demographics**: Younger age group (18-35), mostly single or recently married
- **Professional**: Entry-level to mid-career professionals, often artists or engineers
- **Behavior**: Moderate spending score, smaller family sizes (1-3 members)
- **Key Characteristics**: Career-focused, likely recent graduates, building work experience

Segment B - The Established Families

- **Demographics**: Middle-aged (36-50), mostly married with children
- **Professional**: Stable careers with moderate work experience (5-10 years)
- **Behavior**: Balanced spending patterns, medium family sizes (3-4 members)
- **Key Characteristics**: Family-oriented, financially stable, established professionals

Segment C - The High Spenders

- **Demographics**: Mixed age range but with higher proportion of older customers
- **Professional**: Varied professions but includes high-income occupations
- **Behavior**: Highest spending scores, larger family sizes possible
- **Key Characteristics**: Affluent customers, likely higher income levels, premium preferences

Segment D - The Conservative Consumers

- **Demographics**: Broad age range, largest segment by volume
- **Professional**: Diverse professions, includes many artists and service workers
- **Behavior**: Lowest spending scores, cautious consumption patterns
- **Key Characteristics**: Value-conscious, practical buyers, largest customer base

Segment Comparison:

- **Age Progression**: A (youngest) → B (middle) → C/D (mixed but older)
- **Spending Pattern**: C (highest) → B (medium) → A (moderate) → D (lowest)
- **Family Dynamics**: B (family-focused) → C (mixed) → A/D (smaller families)
- **Career Stage**: A (early) → B (established) → C/D (varied)

14. Business Insights and Reflection

Business Insights and Reflection

1. Which features appear to differentiate customer segments the most?

- **Primary Differentiators**: Spending Score, Age, and Profession are the strongest predictors
- **Secondary Differentiators**: Work Experience, Family Size, and Marital Status
- **Weak Differentiators**: Gender and Var_1 (anonymized category)
- **Key Insight**: Customer behavior (spending) and demographics (age/profession) are more important than basic demographics alone

2. Which segment seems to have the highest spending score?

- **Segment C** has the highest spending scores based on the analysis
- These are likely affluent customers who prefer premium products/services
- Business opportunity: Target Segment C with high-value offerings and premium experiences

3. Do younger customers appear to spend more?

- **No, younger customers (Segment A) show moderate spending patterns**
- Segment C (mixed age but includes older customers) has the highest spending

- **Insight**: Spending power increases with age and career progression, not necessarily youth

4. Does profession appear related to segmentation?

- **Yes, profession is strongly related to segmentation**
- Artists are concentrated in specific segments (likely A and D)
- Engineers and high-income professions appear in Segment C
- **Business implication**: Professional background is a reliable indicator of customer behavior and preferences

5. What additional preprocessing or modeling would you do next?

Preprocessing Improvements:

- Feature engineering: Create interaction terms (Age × Profession, Work Experience × Spending Score)
- Outlier treatment: Consider capping extreme values while preserving business insights
- Feature selection: Use techniques like recursive feature elimination

Modeling Approaches:

- **Classification Models**: Random Forest, Gradient Boosting, SVM for segmentation prediction
- **Clustering**: K-means or hierarchical clustering to validate/refine segments
- **Ensemble Methods**: Combine multiple models for better accuracy

Advanced Techniques:

- Dimensionality reduction (PCA) for visualization and feature compression
- Cross-validation to ensure model robustness
- Hyperparameter tuning for optimal performance

Business Application:

- Develop targeted marketing campaigns for each segment
- Create personalized product recommendations
- Optimize pricing strategies based on segment characteristics
- Monitor segment evolution over time for strategy adjustments

15. Save the Cleaned Dataset (Optional)

In [39]:
```
# Uncomment to save
df_clean.to_csv('customer_segmentation_cleaned.csv', index=False)
print('Cleaned dataset saved.')
```

```
Cleaned dataset saved.
```